

Jagiellonian University
Department of Theoretical Computer Science

Jakub Janór

Implementation of separator-based algorithms for planar graphs

Bachelor's Thesis

Supervisor: dr hab. inż. Krzysztof Turowski

June 2026

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Definitions and important lemmas	3
1.2.1	Planar graphs	5
1.2.2	Hard problems and planar graphs	9
1.2.3	Approximation algorithms	10
2	Planar Separator Theorem	11
2.1	Planar Separator Algorithm	11
2.1.1	BFS layering	12
2.1.2	Fundamental cycle	13
2.1.3	Layering	15
2.1.4	Main theorem	16
2.2	Divide and Conquer Approach	20
3	Applications of planar separator theorem	25
3.1	Matching algorithm	25
3.1.1	Algorithm	26
3.1.2	Correctness proof	26
3.1.3	Time complexity proof	29
3.2	Vertex cover algorithm	29
3.2.1	Algorithm	30
3.2.2	Correctness of the algorithm	32
3.2.3	Time complexity	35
4	Benchmarks and implementation details	36
4.1	Implementation details	36
4.1.1	KOALA NetworKit	36
4.1.2	Lipton-Tarjan planar separator algorithm	37
4.1.3	MISP algorithm	37
4.1.4	Vertex Cover algorithm	38

4.1.5	Maximum Matching algorithm	38
4.2	Benchmark structure and results	38

Chapter 1

Introduction

1.1 Motivation

The planar separator theorem is a fundamental result in graph theory that states that any planar graph can be separated into two roughly equal parts by removing a small number of vertices. One of the first times the concept of a separator was brought to a broader audience was in [18], where Jordan proved that every free tree T has a separator of size 1 (a single vertex), whose removal partitions T into two or more connected components each of size at most $n/2$. The separator theorem for planar graphs that will be discussed in this paper has numerous applications in various fields, including computer science, engineering, and mathematics, serving as the baseline for designing highly efficient divide and conquer algorithms. The main idea of this thesis is to bridge the gap between the theoretical guarantees of the planar separator theorem and practical performance of the algorithms:

- In Section 1.2 we introduce a few definitions that will be needed for later content. Moreover we will discuss properties of planar graphs that make them special among other graph classes.
- In the second chapter we present the outline of the proof for Lipton and Tarjan planar separator theorem with addition to a few lemmas that will be the cornerstone for developing approximation algorithms in the third section.
- In the third chapter we elaborate on the practical consequences of all the lemmas developed in the previous section. We shall explore two algorithms for two different problems: maximum matching problem for which we obtain speedup and minimum vertex cover problem for which we obtain better approximation ratio than in general case.

- In the last chapter we focus on practical implementations of the stated algorithms, and see how their performance scales with different topological families of planar graphs and different sizes.

1.2 Definitions and important lemmas

Definition 1.1 (graph, [9]). *Graph G is a tuple (V, E) , where V is called a vertex set and E are 2-element subsets of V .*

In this work we only deal with undirected graphs and for simplicity we shall define $(u, v) \in E$, where $u, v \in V$ as an undirected edge.

Definition 1.2 (incidency, adjacency, [9]). *A vertex $v \in G$ is called incident to $e \in E$ if $v \in e$. We call two vertices $u, v \in G$ adjacent in G if they are incident to the same edge.*

In many algorithms in this paper we will need to do some pre-processing that in most cases will be based on reducing a graph to a simpler form, without vertices that have a small amount of incident edges. To make notation simpler we can introduce the concept of degree of a vertex.

Definition 1.3 (degree, [9]). *We define a **degree** of a vertex v in a graph $G = (V, E)$ as the number of edges in G that v is incident to and denote it as $\deg(v)$.*

After reducing a graph, we get rid of some vertices and edges incident to them. In the process of such reduction we obtain a *subgraph* of the original graph.

Definition 1.4 (subgraph, [9]). *We call graph $G' = (V', E')$ a subgraph subgraph of $G = (V, E)$ (denoted as $G' \subseteq G$) if $V' \subseteq V$ and $E' \subseteq E$.*

When vertices are removed from a graph, all edges incident to these vertices are removed as well. However, if we only specify the set of remaining vertices, there is a natural way to define the resulting subgraph: we retain every edge of the original graph whose both endpoints remain. Such a subgraph is called an *induced subgraph*. This distinction will be useful when describing the graphs obtained during the preprocessing steps.

Definition 1.5 (induced subgraph, [9]). *Let $G = (V, E)$ be a graph and let $V' \subseteq V$. A subgraph $G' = (V', E')$ of G is called the subgraph **induced** by V' if for every pair of vertices $u, v \in V'$, the edge $(u, v) \in E'$ if and only if $(u, v) \in E$. This subgraph is denoted by $G[V']$.*

It is worth noting that degrees of vertices in an induced subgraph are different than in the original graph. If $G = (V, E)$ and $H = G[V']$ is an induced subgraph for $V' \subseteq V$, then for $v \in H$ we can define $\deg_H(v)$ as the number of edges in H that v is incident to. This notation emphasises that we are only considering the edges within the subgraph H , as opposed to the vertex's original degree in G .

Another concept that is connected with subgraphs is a spanning subgraph, that will be used later when talking about spanning trees.

Definition 1.6 (spanning-subgraph, [9]). *Let $G = (V, E)$ be a connected graph. We call $G' \subseteq G$ a **spanning subgraph** of G if $V' = V$.*

In order to consider separators, we have to introduce the concept of connectivity at first.

Definition 1.7 (connectivity, [9]). *Let $G = (V, E)$ be a graph. We call G **connected** if for each $u, v \in V$ there exists a path in G connecting u and v .*

As the name suggests a separator, separates the sets of vertices from each other, meaning there is no edge connecting any two vertices between them. After removal of a separator, the graph is being disconnected. Subgraphs that are left afterwards are called connected components.

Definition 1.8 (connected components, [9]). *Let $G = (V, E)$ be a graph. A subgraph $G' = (V', E')$ is called a **connected component** of G if G' is the subgraph induced by V' , G' is connected, and V' is a maximal subset of V with respect to this property.*

After some basic graph theoretical definitions, we move on to consider different classes of graphs, that will appear in this paper.

One of the simplest classes of graphs are paths, which is just a sequence of distinct vertices connected by edges.

Definition 1.9 (path, [9]). *A path is a non-empty graph $P = (V, E)$ of the form $V = \{x_0, \dots, x_k\}$, $E = \{(x_0, x_1), \dots, (x_{k-1}, x_k)\}$ where the x_i are distinct.*

If we connect the first and last vertex of a path with an edge, we create a cycle (complete a cycle with that edge).

Definition 1.10 (cycle, [9]). *If $P = (V, E)$ is a path of the form $V = \{x_0, \dots, x_k\}$, $E = \{(x_0, x_1), \dots, (x_{k-1}, x_k)\}$ and $k \geq 3$, then cycle is the graph $C = (V, E \cup \{(x_k, x_0)\})$.*

After defining a cycle, we naturally may consider a graph that does not have any cycles.

Definition 1.11 (forest, tree, [9]). *Let $G = (V, E)$ be a graph. We call G a **forest** if it does not contain any cycles (i. e. is acyclic). We call a connected forest a **tree** T .*

We say that a tree is *minimally connected*. It means that if we remove any edge, it will become disconnected. Also trees are *maximally acyclic*, which means that if we add any edge to this graph, it will contain a cycle. One more important property of a tree is the fact, that between any two vertices, there exists one unique path.

Another important class of graphs that will show up when talking about a Vertex Cover Problem in Chapter 3 is a bipartite graph.

Definition 1.12 (bipartite, [9]). *Let $G = (V, E)$ be a graph. We call G a **bipartite** graph if there exists a partition of V into two subsets X, Y such that $X \cap Y = \emptyset$ and no two vertices in the same subset are adjacent in G . To indicate that a graph is a bipartite graph we will use notation $G = B(X, Y, E)$, where X, Y, E are defined as above.*

Another concept that will be useful when discussing planar graphs and their relationship with plane graphs is that of graph isomorphism.

Definition 1.13 (homomorphism, isomorphism, [9]). *Let $G = (V, E)$ and $G' = (V', E')$. We call a map $\phi: V \rightarrow V'$ a **homomorphism** if $(\phi(u), \phi(v)) \in E'$ whenever $(u, v) \in E$. If ϕ is a bijection and ϕ^{-1} is also a homomorphism, then ϕ is an **isomorphism**. We call G and G' **isomorphic** if there exists a map $\phi: V \rightarrow V'$ that is an isomorphism.*

An isomorphism provides a correspondence between the vertices of two graphs that preserves their structure. Thus, isomorphic graphs may have different representations or descriptions while retaining the same graph structure.

1.2.1 Planar graphs

After basic definitions for undirected graphs in general, we shall move on to definitions specific to planar graphs which are the class of graphs that we are interested in this paper. However at first we shall start with the definition of *plane* graphs.

Definition 1.14 (plane graphs, [9]). A **plane graph** is a pair (V, E) of finite sets with the following properties:

1. $V \subseteq \mathbb{R}^2$
2. every edge in E is an arc between two vertices from V
3. different edges have different sets of endpoints
4. the interior of an edge contains no vertex and no point of any other edge

A plane graph defines a graph G on (V, E) in a natural way.

Definition 1.15 (faces, [9]). Let $G = (V, E)$ be a plane graph. We call the regions of $\mathbb{R}^2 \setminus G$ the **faces** of G .

Since G is bounded, exactly one face of G is unbounded. This face is the outer face of G , while all other faces are called inner faces. We denote a set of faces of G as $F(G)$. Now we can see that because of the specific definition of plane graphs they have some really interesting properties that no other class has. The famous theorem for plane graph is the Euler's formula, that connects the number of vertices, edges and faces of a plane graph in one equation.

Theorem 1.2.1 (Euler's Formula, [9]). Let $G = (V, E)$ be a connected plane graph with faces F . Number of edges, vertices and faces of G is connected by the following formula:

$$|V| - |E| + |F| = 2$$

To obtain a very important lemma, which bounds the number of edges in a plane graph by the number of vertices, we have to introduce one more lemma.

Lemma 1.2.2 (maximal plane, [9]). A plane graph $G = (V, E)$ with at least 3 vertices is maximally plane (it is impossible to add any more edges without violating planarity) when it is the plane triangulation (i.e. each face is a triangle - a cycle of length 3 on plane);

Now we can finally introduce:

Corollary 1.2.2.1 ([9]). A plane graph $G = (V, E)$ with at least 3 vertices has at most $3|V| - 6$ edges. Every plane triangulation has exactly $3|V| - 6$ edges.

Proof. By Lemma 1.2.2 we can prove only the second part of the Corollary, and the first part naturally follows (as plane triangulation is maximally plane, so adding more edges would destroy planarity). Let G be a plane triangulation. At first we shall see that if each face is a triangle, then if we can see that each face contributes exactly 3 edges to the total number of edges. On the other hand if we look at it from the perspective of an edge, we can see that each edge is incident to exactly two faces. By that double counting argument we have that $2|E| = 3|F|$, so $|F| = 2/3|E|$. By putting that into Euler's formula we get $|V| - 1/3|E| = 2$, which is exactly $|E| = 3|V| - 6$. $\square$

We shall see that this is a very important property of plane graphs, as they are sparse graphs, so each $O(|V| + |E|)$ algorithm on them is an $O(|V|)$ algorithm. Now we shall look at the concept of a minor of a graph and see a property of plane graphs that identifies them among other classes of graphs.

Definition 1.16 (edge contraction, [9]). *Let $G = (V, E)$ be a graph and let $(u, v) \in E$ be an edge. We call **edge contraction** of (u, v) an operation which transforms G into G' by merging vertices u and v into one vertex v' , such that it is adjacent in G' to all the vertices that u and v were adjacent to in G . We also remove all the loops and multi edges that are created in such operation.*

This edge contraction definitions leads to a definition of a minor.

Definition 1.17 (minor, [9]). *Let $G = (V, E)$ be a graph. A graph H is a **minor** of G (denoted by $H \preceq G$) if it is isomorphic to a graph G' that can be obtained from G in a finite sequence of vertex deletions, edge deletions and edge contractions.*

With definitions of edge contraction and minors comes a following lemma, that will be important later on during discussion of Lipton Tarjan planar separator algorithm.

Lemma 1.2.3 ([19]). *Let G be any planar graph. Shrinking any connected subgraph of G to a single vertex preserves planarity.*

By using Corollary 1.2.2.1 we can perform a simple test to filter out the graphs that are not plane graphs. The example of such a graph is K_5 (a maximal graph on 5 vertices). It has 5 vertices and 15 edges, and by putting into formula we get $10 \leq 9$ which is a contradiction. Interestingly an even tighter bound than the one from Corollary 1.2.2.1 can be found for bipartite graphs.

Corollary 1.2.3.1. *A bipartite plane graph $G = (V, E)$ with at least 4 vertices has at most $2|V| - 4$ edges.*

Proof. In each bipartite graph each face is bounded by a cycle of length at least 4. So by counting faces, each one contributes at least 4 edges. On the other side each edge is incident to exactly 2 faces. So by double counting we have $2|E| \geq 4|F|$, so $|F| \leq 1/2|E|$. By putting that into Euler's formula we obtain $|V| - 1/2|E| \geq 2$ and thus $|E| \leq 2|V| - 4$. $\square$

Using this corollary we can show that $K_{3,3}$ (a maximal bipartite graph with sets of sizes 3 and 3) is not a plane graph. Interestingly these two graphs (K_5 and $K_{3,3}$) are the biggest obstacles for a graph to be a plane graph. This fact is expressed by the following famous theorem.

Theorem 1.2.4 (Wagner's theorem, [27]). *Graph $G = (V, E)$ is a plane graph if and only if its minors include neither K_5 nor $K_{3,3}$.*

We shall see that this theorem is much stronger than what I wrote earlier. Not only are K_5 and $K_{3,3}$ biggest obstacles for a graph to be a plane graph. Actually the minors of those graphs are the only obstacles. All those properties were shown for plane graphs. Now we can complete everything by defining what a planar graph is.

Definition 1.18 (planar embedding, [9]). *The **planar embedding** of a graph $G = (V, E)$ in a plane is an isomorphism of G with any plane graph H .*

In later sections a very important part will be computing the embedding quickly. Embedding specifies a cyclic order on edges incident to each vertex, corresponding to their order around the vertex in a drawing. From algorithmic perspective it is essential, as it allows to handle geometrical properties of planar graphs programmatically. Thus it being an essential part of the algorithm, it is important to compute it quickly, ideally in linear time. Luckily, there are several algorithms that do just that (see Table 1.1). One of the first ones that did in that quickly was Hopcroft-Tarjan algorithm [17], but the most widely used, and the one that will be used in this paper is Boyer-Myrvold algorithm [5].

Definition 1.19 (Planar Graph). *Graph $G = (V, E)$ is called **planar** if there exists a planar embedding that maps it to a plane graph H .*

Table 1.1: Selected linear algorithms for planar embedding.

Authors	Year	Original work
Hopcroft and Tarjan	1974	[17]
Booth and Lueker	1976	[4]
Chiba, Nishizeki, Abe, and Ozawa	1985	[6]
Boyer and Myrvold	1999	[5]

1.2.2 Hard problems and planar graphs

From the previous section we can see that planar graphs are a very special class of graphs. Some of their properties actually help reduce the time complexity for some problems by a large margin. The example is maximum-clique problem which asks to find the biggest clique in a graph. For any abstract graph, that problem is NP-complete. However for planar graphs it can be done in $O(|V|)$ time. It is due to the fact that the maximum size of a clique in planar graphs is 4, which comes directly from Theorem 1.2.4. The algorithm for that problem is based on the following lemma.

Lemma 1.2.5. *Let $G = (V, E)$ be a planar graph. There exists a vertex v such that $\deg(v) \leq 5$.*

Proof. Let's suppose that the minimum degree of G is 6. Then we can once again construct a double counting argument: each edge is incident to exactly two edges, and each vertex is incident to at least 6 edges. So we have $2|E| \geq 6|V|$, then $|E| \geq 3|V|$. Now we can use Corollary 1.2.2.1 and conclude that $3|V| \leq 3|V| - 6$ which leads to contradiction. $\square$

Another application of this lemma is Graph Coloring problem which is NP-complete in general case.

Definition 1.20 (vertex coloring, [9]). *A vertex coloring of a graph $G = (V, E)$ is a map $c: V \rightarrow S$ such that $c(v) \neq c(w)$ whenever v and w are adjacent. The element of the set S are called colors. Graph Coloring problem is finding the minimum size S such that c is a correct coloring.*

Because of Lemma 1.2.5 we can easily prove that for planar graphs it is sufficient to use 6 colors. However, that is not the smallest number possible. In 1977 it was proven by Haken and Appel [1] that each planar graphs is 4-colourable, moreover it can be done in polynomial time. And although most of the NP-complete problems are still in this complexity class for planar graphs (for example determining whether a planar graph is 3-colourable is still NP-complete [14]), there is another opportunity to reduce the time of

computation. We can resort to approximation algorithms that will utilize planar separator theorem, that is the main topic of this paper.

1.2.3 Approximation algorithms

When we can exchange the optimality of the solution for the problem to gain some computation time, we shall take into consideration approximation algorithms. The idea of such an algorithm is that the found solution will be worse than the optimal by some factor, in exchange for better time complexity. In some cases one can find a quicker algorithm for a polynomial problem, but sometimes we can find a polynomial approximation algorithm to an NP -complete problem which is a great improvement in terms of time complexity.

Definition 1.21. [28] *An α -approximation algorithm for an optimization problem is a polynomial time algorithm that for all instances of a problem produces a solution whose value is within a factor of α of the value of an optimal solution.*

For an α -approximation algorithm, we shall call α the approximation ratio, and $1 - \alpha$ the approximation error. For example, the 4-coloring algorithm for planar graphs described in the previous section yields a $4/3$ -approximation algorithm for the minimum vertex coloring problem.

We can also look for something better than a constant approximation ratio. There are some problems for which we can get a solution arbitrarily close to the optimal solution. These problems have polynomial-time approximation schemes.

Definition 1.22. *A polynomial-time approximation scheme is a family of algorithms $\{A_\epsilon\}$, where there is an algorithm for each $\epsilon > 0$, such that A_ϵ is a $(1 + \epsilon)$ -algorithm (for minimization problems) or $(1 - \epsilon)$ -algorithm (for maximization problems).*

In Chapter 3 we shall explore how planar separators can be utilized to create approximation schemes for two problems: an already polynomial-time problem of finding maximum matching for which we shall find an $O(n \log n)$ algorithm, and NP -complete problem of finding a minimum vertex cover.

Chapter 2

Planar Separator Theorem

2.1 Planar Separator Algorithm

Let us introduce first the central concept of this work: a separator.

Definition 2.1. *A set $S \subseteq V(G)$ is a separator if induced graph $G^* = G - S$ has at least two connected components $A, B \subseteq G^*$.*

Through the years, more and more classes of graphs started to show some interesting properties regarding vertex separators, so a general theorem was assembled, which is called a $f(n)$ -separator theorem:

Theorem 2.1.1. *[19] Let $\mathcal{S}$ be a class of graphs closed under a subgraph relation (i.e. if $G_1 \in \mathcal{S}$ and G_2 is a subgraph of G_1 , then $G_2 \in \mathcal{S}$). There exist constants $\alpha < 1$, $\beta > 0$ such that if G is any n -vertex graph in $\mathcal{S}$, the vertices of G can be partitioned into three sets A, B, C such that no edge connects any vertex in A and B , neither A nor B have more than αn vertices and C contains no more than $\beta f(n)$ vertices*

Such theorem for planar graphs, where $f(n) = O(\sqrt{n})$, $\alpha = 2/3$, $\beta = 1$ is called **planar separator theorem**.

First breakthrough in that area was in 1951 [26], when Ungar proved a slightly weaker theorem with $f(n) = O(\sqrt{n} \log^{3/2} n)$. A small improvement to that was later achieved in 1973 by George [15] where he proved that any grid graph of size n has a $O(\sqrt{n})$ separator. However, the first to prove planar separator theorem in the full form were Lipton and Tarjan in 1977 [19], where it was proven that for each planar graph a separator of size at most $2\sqrt{2}\sqrt{n}$ exists. In many articles later after Lipton and Tarjan, the constant β was reduced, however its lower bound is $(\sqrt{4\pi\sqrt{3}})/3$ which was proven by Djidjev in [10]. Djidjev established this absolute limit not by looking at flat grids,

but by constructing worst-case planar graphs based on the triangulation of a sphere, proving mathematically that no algorithm can separate these dense spherical meshes with fewer vertices.

The main theorem, this whole thesis and all algorithms are based on is the planar separator theorem introduced by Lipton and Tarjan. What is really interesting about this theorem is that it holds even for weighted graphs, with weights (also called costs) assigned to vertices. However, at first we shall make a formal definition for that generalization.

Definition 2.2. *Weighted graph G is a tuple (V, E, w) , where V is called a vertex set, E are 2-element subsets of V (i. e. (V, E) is a graph) and $w: V \rightarrow \mathbb{R}^+ \cup \{0\}$ is a weight function.*

We can extend the definition of a weight function for a single vertex to a weight function for a whole graph.

Definition 2.3. *Let $G = (V, E, w)$ be a weighted graph. Let $W: G \rightarrow \mathbb{R}^+ \cup \{0\}$ be defined as $W(G) = \sum_{v \in V} w(v)$. We shall call $W(G)$ the cost of a graph G . Moreover, let $X \subseteq V$, we will define $W(X)$ as the cost of a set of vertices X .*

Theorem 2.1.2 (Planar Separator Theorem [19]). *Let $G = (V, E, w)$ be a weighted planar graph with n vertices, such that $W(G) \leq 1$. Then we can partition vertices of G into three sets A , B , and C such that:*

- (a) *there exists no edge $u, v \in E$ with $u \in A$ and $v \in B$,*
- (b) *$W(A) \leq 2/3$ and $W(B) \leq 2/3$,*
- (c) *$|C| \leq 2\sqrt{2}\sqrt{n}$.*

Moreover, such a partition can be found in $O(n)$ time.

To prove this theorem we will base on a few already proven facts on planar graphs as well as some auxiliary lemmas.

2.1.1 BFS layering

One of the most important parts of the proof of Theorem 2.1.2 is the concepts of layers which are levels of a BFS spanning tree.

Definition 2.4. *Let $G = (V, E)$ be a connected planar graph and $t \in V$ be a vertex that we will call the root of a tree. A BFS spanning tree is a tree T rooted in t such that for all $v \in V$ it holds that $d_T(t, v) = d_G(t, v)$.*

Using such a spanning tree we can partition the vertex set V into disjoint sets that we will call layers (or levels). So we can define it as $L_i = \{u \in V : d(t, u) = i\}$ (clearly, $L_0 = \{t\}$). Such a layering has two important properties that will be very useful in the future.

Lemma 2.1.3. *If $(u, v) \in E$ and $u \in L_i, v \in L_j$, then $|i - j| \leq 1$.*

The proof for this lemma is very simple by following the construction of BFS spanning tree during BFS traversal. The other property is also very intuitive and follows directly from Lemma 2.1.3.

Lemma 2.1.4. *For any $l \in \{1, \dots, r - 1\}$ removal of L_l from G separates it into two disconnected components $\bigcup_{i=0}^{l-1} L_i$ and $\bigcup_{i=l+1}^r L_i$*

It is easy to see that this lemma shows connection between BFS layering and separation of components, and we shall see how it is incorporated into the proof for the main theorem in near future. Another separating property can be derived for planar graphs specifically.

Theorem 2.1.5. *[19] Let C be any closed curve in the plane. Removal of C divides the plane into exactly two connected regions: "inside" and "outside" of C .*

Although the proof for this theorem is pretty formal and uses topological properties of the plane, intuition for it is pretty straightforward. As the edges in a planar graphs cannot cross on the plane, a closed curve is a natural separator, as no edge from a vertex on *outside* can cross the curve to reach any vertex on *inside*.

2.1.2 Fundamental cycle

The first lemma needed to prove Theorem 2.1.2 shows us how the intuitive separation property from Theorem 2.1.5 can be used to get the needed costs of separated subgraphs. To do this we will need the concept of a fundamental cycle.

Definition 2.5. *Let $G = (V, E)$ be a graph and let T be a spanning tree of G . For any non-tree edge $e = (u, v) \in E \setminus E(T)$, the unique simple cycle consisting of e and the unique path in T connecting u and v is called the **fundamental cycle** defined by e .*

As we can see from this definition, fundamental cycles can be defined if we have a spanning tree for a graph. It connects with BFS layering defined earlier, and in the next section we will explore this further. However, we begin with the lemma related to fundamental cycles.

Lemma 2.1.6. [19] *Let $G = (V, E, w)$ be a weighted planar graph with n vertices, such that $W(G) \leq 1$. Suppose that G has a spanning tree T with radius r . Vertices of G can be partitioned into three sets A, B, C such that:*

- (a) *there exists no edge $u, v \in E$ with $u \in A$ and $v \in B$,*
- (b) *$W(A) \leq 2/3$ and $W(B) \leq 2/3$,*
- (c) *$|C| \leq 2r + 1$ and C contains root of T .*

Proof. We shall see that if G has a vertex v with cost at least $1/3$, the lemma is true. To prove that, let us consider G' to be the path from v to the root of T and C its vertex set. We can see that it has less than $2r + 1$ vertices and contains the root. Moreover: $W(C) \geq 1/3$ so if we assign $A = V \setminus C$ and let $B = \emptyset$ then the proof is completed.

Thus let's assume that G does not have a vertex with cost at least $1/3$. Let's now embed G in plane using any algorithm (See discussion below Definition 1.18), and make that graph maximal planar by adding edges, so that each face is a triangle.

Now each one of the non-tree edges, that is those added when making G maximal as well, creates a cycle C with tree edges, its length being at most $2r + 1$. From Theorem 2.1.5 we know that C splits the plane into two parts: the *inside* and *outside*. If we can prove that there exists a cycle C such that neither side contains vertices whose total cost exceeds $2/3$, then the lemma is proven.

Let (x, z) be a non-tree edge that minimizes the maximum cost of either sides of the cycle C' that it closes. If there are many edges with this property, pick the one with the smallest number of faces on the maximized side. Let's say without loss of generality, that the *inside* has the cost at least as high as the *outside*. If that cost does not exceed $2/3$ then the lemma is true. Otherwise let's consider a triangle F_T inside C' such that (x, z) is one of its boundaries. Let y be the third vertex of F_T . Now we have to do case analysis based on properties of the edges of F_T :

- if both $(x, y), (y, z)$ lie on the cycle then $F_T = C'$, which contradicts with the fact that inside has a positive cost, so it has to have vertices
- if one of $(x, y), (y, z)$ (assume without loss of generality it is (x, y)) lies on the cycle, then (y, z) is a non-tree edge (it has to be a non-tree edge, as otherwise it would close a smaller cycle (x, y, z) , which is not possible as (x, z) defines C') that defines a new cycle C'' with the same cost, but one less face than C' which contradicts with choice of (x, z) .

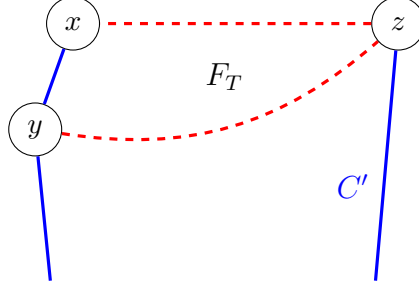


Figure 2.1: See how, (y, z) defines a new cycle with one fewer face than C' .

- if none of $(x, y), (y, z)$, we have to make another case analysis looking at all possible configurations of those two edges being a tree edge. All those configurations lead to a contradiction (a complete proof is to be found in [19]).

As we can see, all of the cases lead to contradiction, so the cost inside C' has to be at most $2/3$. $\square$

2.1.3 Layering

As forecasted earlier, in this section we will show how we can utilize facts introduced in Section 2.1.1 and just proved Lemma 2.1.6 to show the construction of spanning tree T and graph partition.

Lemma 2.1.7. [19] *Let $G = (V, E, w)$ be a planar graph with n vertices, such that $W(G) \leq 1$. Suppose that vertices are partitioned into layers using a BFS spanning tree T rooted in t and with radius r . Let $r+1$ be an additional level with no vertices. Given any two levels l_1, l_2 such that $\bigcup_{i=0}^{l_1-1} w(L_i) \leq 2/3$ and $\bigcup_{i=l_2+1}^{r+1} w(L_i) \leq 2/3$, it is possible to find a partition of G into sets A, B, C such that:*

- (a) *there exists no edge $u, v \in E$ with $u \in A$ and $v \in B$,*
- (b) *$W(A) \leq 2/3$ and $W(B) \leq 2/3$,*
- (c) *$|C| \leq |L_{l_1}| + |L_{l_2}| + \max\{0, 2(l_2 - l_1 - 1)\}$.*

Proof. Let's assume that $l_1 \geq l_2$. Then we can select $A = \bigcup_{i=0}^{l_1-1} L_i$, $B = \bigcup_{i=l_1+1}^{r+1} L_i$ and $C = L_{l_1}$. We can see that the lemma is true, as by definition $W(A) \leq 2/3$ and $|C| = |L_{l_1}| \leq |L_{l_1}| + |L_{l_2}| + \max\{0, 2(l_2 - l_1 - 1)\}$. Moreover, because $l_1 \geq l_2$, then $B \subseteq \bigcup_{i=l_2+1}^{r+1} L_i$, so $W(B) \leq \bigcup_{i=l_2+1}^{r+1} w(L_i) \leq 2/3$.

Thus suppose $l_1 < l_2$. Let's define sets: $D_1 = \bigcup_{i=0}^{l_1-1} L_i$, $D_2 = \bigcup_{i=l_1+1}^{l_2-1} L_i$, $D_3 = \bigcup_{i=l_2+1}^{r+1} L_i$, all of which may be empty. We can see that they are disjoint by Lemma 2.1.4. The only set that can have cost exceeding $2/3$ is D_2 (by assumptions of this lemma). If $W(D_2) \leq 2/3$, then the lemma is true, as we can set the most costly part to be A , B will consist of the other two parts and $C = L_{l_1} \cup L_{l_2}$.

Let's then suppose that the cost of D_2 is greater than $2/3$. Delete all vertices on levels l_2 and above, and contract levels $0, \dots, l_1$ into one vertex t' with cost 0. From Lemma 1.2.3 we know that contracting vertices preserves planarity. We can now see that the new graph H has a spanning tree rooted in t' and radius $l_2 - l_1 - 1$.

Apply Lemma 2.1.6 to H and let A_H, B_H, C_H be the resulting partition. Let A be the set among A_H and B_H that has greater cost and let C consist of all of the vertices in L_{l_1}, L_{l_2} in the original graph G as well as all of the vertices from $C_H - \{t'\}$. And finally B will consist of all of the vertices from V that are not in A or C . We can see that it works because by Lemma 2.1.6, the cost of A does not exceed $2/3$. Moreover because of our construction, C has the size of at most $|L_{l_1}| + |L_{l_2}| + 2(l_2 - l_1 - 1)$. However we can see that the cost of $A \cup C_H$ has to be at least $1/3$, as A was the set with greater cost among A_H, B_H , hence $W(B) \leq 2/3$. $\square$

2.1.4 Main theorem

Now we have all of the tools to tackle Theorem 2.1.2.

Proof. Assume at first that G is connected. At first we partition into layers using BFS layering getting a spanning tree T rooted in t with radius r . Additionally we define two empty boundary levels L_{-1} and L_{r+1} . Let us define l_1 as the number of the level such that the cost of vertices included in levels 0 through $l_1 - 1$ is strictly less than $1/2$ and the cost of vertices in levels 0 through l_1 be at least $1/2$.

If no such level exists then the cost of vertices is less or equal than $1/2$, and such we can assign $A = V$, $B = C = \emptyset$, thus the theorem is true.

We shall define $k = \sum_{i=0}^{l_1} L(i)$. Now, we find two levels:

- l_0 such that $l_0 \leq l_1$ and $|L_{l_0}| + 2(l_1 - l_0) \leq 2\sqrt{k}$,
- l_2 such that $l_1 + 1 \leq l_2$ and $|L_{l_2}| + 2(l_2 - l_1 - 1) \leq 2\sqrt{n - k}$.

If l_0, l_2 exist then by Lemma 2.1.7 (as by definition of l_1 and thus l_0, l_2 conditions for it hold) we have a partition A, B, C and the cost of C does not exceed $2(\sqrt{k} + \sqrt{n - k})$.

Now by employing Cauchy-Schwartz inequality:

$$(u_1v_1 + u_2v_2)^2 \leq (u_1^2 + u_2^2)(v_1^2 + v_2^2)$$

with $u^T = [2, 2]$ and $v^T = [\sqrt{k}, \sqrt{n-k}]$ we can deduce that the cost of C is bounded by $2\sqrt{2}\sqrt{n}$.

Now we shall see that such levels l_0 and l_2 exist. Let's suppose that l_0 does not exist. Then for $\forall_{i \leq l_1} |L_i| \geq 2\sqrt{k} - 2(l_1 - i)$. As $|L_0| = 1$ we have $1 \geq 2\sqrt{k} - 2l_1$ and thus $l_1 + 1/2 \geq \sqrt{k}$. If we take the floor of that we have that $l_1 \geq \lfloor \sqrt{k} \rfloor$. And now by definition of k we have:

$$\begin{aligned} k &= \sum_{i=0}^{l_1} L(i) \stackrel{(1)}{\geq} \sum_{i=l_1-\sqrt{[k]}}^{l_1} L(i) \stackrel{(2)}{\geq} \sum_{i=l_1-\sqrt{[k]}}^{l_1} 2\sqrt{k} - 2(l_1 - i) \\ &\stackrel{(3)}{\geq} (4\sqrt{k} - 2\sqrt{[k]})(\sqrt{[k]} + 1)/2 \geq \sqrt{[k]}(\sqrt{[k]} + 1) > k \end{aligned}$$

Inequality (1) comes from the fact that we can restrict the range over which we sum as $l_1 + 1/2 \geq \sqrt{k}$. Inequality (2) stems from the fact that $L(i) \geq 2\sqrt{k} - 2(l_1 - i)$. And inequality (3) comes from the arithmetic sum formula. And thus we can see a contradiction, so l_0 exists.

An analogous, symmetric proof can be given for l_2 existence. This concludes proof for a connected graph.

If G is not connected, let $G_1, \dots, G_k$ be connected components of G . If no connected component has cost exceeding $1/3$, then let i be the minimum index such that the cost of $V(G_1), \dots, V(G_i)$ exceeds $1/3$. Then we can set $A = V(G_1) \cup \dots \cup V(G_i)$, $B = V(G_{i+1}) \cup \dots \cup V(G_k)$ and $C = \emptyset$ and find our partition. When there is some component with cost at least $1/3$ and not exceeding $2/3$ let's say G_j , then we can set $A = V(G_j)$, $B = V(G) \setminus V(G_j)$, $C = \emptyset$ and find our partition. If there is some component with cost at least $2/3$ then we can employ for that connected component an argument in the first section of this proof to find a partition A^*, B^*, C^* . We can then set $C = C^*$, A will be the vertex set among A^* and B^* that has the greater cost and B shall be the rest of the vertices in the graph. It is easy to see that this concludes our proof. $\square$

$O(n)$ Algorithm

Following the constructions used in order to prove Theorem 2.1.2, Lipton and Tarjan developed an $O(n)$ algorithm to find a partition described in it. On the input we get a planar graph G for example in adjacency lists representation

1. Find a planar embedding of G using an efficient $O(n)$ algorithm (see discussion following Definition 1.18).
2. Find connected components of G and calculate costs of each one. If there is no connected component with cost at least $2/3$, then follow the last part of the proof for Theorem 2.1.2. Otherwise, go to the next step.
3. Find a BFS spanning tree of the most costly component. Compute the level of a vertex and for each level compute $|L_l|$.
4. Find l_1 such that the cost of levels $0, \dots, l_1 - 1$ does not exceed $1/2$ and the cost of levels $0, \dots, l_1$ is greater or equal that $1/2$. Now find the highest level $l_0 \leq l_1$ such that $L(l_0) + 2(l_1 - l_0) \leq 2\sqrt{k}$ and lowest level $l_2 \geq l_1 + 1$ such that $L(l_2) + 2(l_2 - l_1 - 1) \leq 2\sqrt{n - k}$.
5. Delete all vertices on level l_2 and above. Construct a new vertex x and shrink all the levels from 0 to l_0 to that one vertex, rewiring the edges on the boundary of l_0 level to x ¹.
6. Construct a BFS spanning tree T_x rooted in x . Keep track for each vertex v of its parent in T_x and the total cost of descendants of v including v itself. Make the graph maximal by triangulation.
7. Choose any non-tree edge (v_1, w_1) and by following parent pointers find a corresponding fundamental cycle C_1 . Compute costs of either sides of the cycle.
8. By Theorem 2.1.5 we know that a cycle splits the plane into two sides: inside and outside. Let's call the set of vertices on the inside X_I and the set of vertices outside X_O . Using Algorithm 1 (with input being C_1 found in the previous step) find a fundamental cycle C such that $W(X_I) \leq 2/3$ and $W(X_O) \leq 2/3$.
9. Use the cycle found in the previous step and levels found in step 4 to construct a vertex partition as described in proof for Lemma 2.1.7. Extend the partition found for the most costly connected component to entire graph like in the proof for Theorem 2.1.2.

¹For details, see [19] or `LiptonTarjanPlanarSeparator.cpp` from Chapter 4.

Time complexity

It is easy to see that all the steps of the algorithm except steps 7 and 8, take $O(n)$ time. So let's prove that bound for latter as well.

Lemma 2.1.8. *Step 7 of Lipton-Tarjan Planar Separator Algorithm takes $O(n)$ time.*

Proof. At first we shall define what is the lowest common ancestor, which in our case will be a vertex reached by following parent pointers from v_1 and w_1 .

Definition 2.6. *Let $T = (G, V)$ be a tree rooted in t . For $v, w \in T$ consider paths $P_v = (V_1, E_1)$ and $P_w = (V_2, E_2)$ such that $V_1 = \{v, \dots, t\}$ and $V_2 = \{w, \dots, t\}$. The lowest common ancestor (**LCA** in short) of v and w is a vertex x , such that $x \in V_1 \cap V_2$ and $d(x, t)$ is the largest among all of the vertices in $V_1 \cap V_2$ (x is the deepest in T among them).*

In step 7 we find the cycle C_1 by finding LCA of v_1 and w_1 which can easily be done in $O(n)$ time by following parent pointers computed in Step 6. The cycle consists of paths from v_1 and w_1 to their LCA and a non-tree edge (v_1, w_1) .

To compute the costs of both sides, for each vertex in the cycle we can look at the embedding. Let's consider vertex $v \in C_1$. The planar embedding gives us a cyclic order on its incident edges. Each one of these edges is on the inside of the cycle, outside of the cycle or on the cycle (by Theorem 2.1.5).

Now we can use precomputed costs of vertices and their subtrees from Step 6 to find the contribution of a subtree hanging of each one of these edges to their respectable side in $O(1)$ time. And as the number of scanned edges is $O(n)$, then the whole step is linear. $\square$

Now we can move on to prove that Step 8, which is described in Algorithm 1 is linear.

Lemma 2.1.9. *Algorithm 1 has time complexity $O(n)$.*

Proof. In each step of the algorithm we remove at least one face of the planar graph, so the number of iterations is bound by $O(n)$.

In each iteration we do some $O(1)$ work, work proportional to the length of path P_T and some work proportional to the number of edges scanned in cycles R_1, R_2 . We can see that if $e \in P_T$, then that computation regarding tree path is done only once, as that edge will in the next iteration be on the boundary of new cycle.

So all the work regarding computations of P_T amortizes to $O(n)$. Regarding scanning, we can see that if we scanned k edges in each cycle before stopping we did computations proportional to $O(k)$. But in the next iteration at least k edges will be outside the next cycles. So those computations amortize to $O(n)$ as well. $\square$

Algorithm 1 FUNDAMENTAL_CYCLE

input: C_1 : cycle

output: C : fundamental cycle

```

1:  $i \leftarrow 1$ 
2:  $C_i \leftarrow C_1$ 
3: Let  $(v_i, w_i)$  be a non-tree edge completing  $C_i$ 
4: while cost inside  $C_i$  exceeds  $2/3$  do
5:   Let  $(v_i, y, w_i)$  be a triangle inside  $C_i$  with boundary edge  $(v_i, w_i)$ 
6:   if  $(v_i, y) \in E(T_x)$  or  $(w_i, y) \in E(T_x)$  then
7:     Let  $(v_{i+1}, w_{i+1})$  be a non-tree edge among  $(v_i, y), (w_i, y)$ 
8:     Let  $C_{i+1}$  be a cycle defined by  $(v_{i+1}, w_{i+1})$ 
9:     Compute cost inside  $C_{i+1}$ 
10:  else
11:    Find tree path  $P_T$  from  $y$  to  $C_i$  path
12:    Let  $z$  be the first vertex in  $P_T$  from  $y$  that is on  $C_i$ 
13:    Compute cost of all vertices on  $P_T \setminus \{z\}$ 
14:    We have two cycles  $R_1$  defined by  $(y, v_i)$  and  $R_2$  defined by  $(y, w_i)$ 
15:    Scan alternatively one edge in  $R_1$  and one in  $R_2$ 
16:    Stop scanning when one of those two cycles have been scanned
17:    Find the cost of both cycles by reusing cost of  $C_i$  and  $P_T \setminus \{z\}$ 
18:    Let  $(v_{i+1}, w_{i+1})$  be the edge among  $(v_i, y), (w_i, y)$  whose cycle has more
    cost inside
19:  end if
20: end while
return  $C_i$ 

```

2.2 Divide and Conquer Approach

Let us recall the divide and conquer approach to solve problems [8]. We can divide the problem into smaller subproblems, solve each subproblem independently, and then combine the results to get the solution to the original problem.

Approximation algorithms for NP-complete problems

If we look at Theorem 2.1.2, we can see that it gives us a way to divide a planar graph into smaller parts efficiently. This is particularly useful for finding approximate solutions to NP-complete problems on planar graphs in polynomial time. Let's look at the following lemma.

Lemma 2.2.1. [7] *Let $G = (V, E, w)$ be a weighted planar graph with n vertices, such that $W(G) \leq 1$ and let $0 < \epsilon < 1$. Then there exists some set C of $O(\sqrt{n/\epsilon})$ vertices such that removal of C from G leaves no connected component with cost more than ϵ . Moreover, such a set can be found in $O(n \log n)$ time.*

Proof. We will consider two cases in here:

If $\epsilon \leq 1/n$, we can simply take $C = V(G)$, which has $O(\sqrt{n/\epsilon})$ vertices and $G - C = \emptyset$, so it has no connected components with cost greater than ϵ trivially.

If $\epsilon > 1/n$, we introduce a simple algorithm:

Algorithm 2 Finding ϵ -planar separator

input: G : planar graph

output: C : separator

```

1:  $C \leftarrow \emptyset$ 
2: while  $G - C$  has a connected component  $K$  such that  $W(K) > \epsilon$  do
3:   Apply Theorem 2.1.2 to  $K$  producing a partition  $(A', B', C')$ 
4:    $C \leftarrow C \cup C'$ 
5: end while
6: return  $C$ 

```

We can see that in each iteration of the while loop of the algorithm, we split the greatest component into two smaller ones, each with cost at most $2/3$ of the original component. Let L be a function that assigns a level to each component that arises during the execution of the algorithm. We can define it as follows: $L(K) = 0$ if K is the component of $G - C$ when the algorithm halts. Otherwise $L(K) = 1 + \max\{L(A_i)\}$, where A_i is the component created by splitting K during the execution of the algorithm.

With such a definition, we can see that for all A, B it holds that if $L(A) = L(B)$ then A and B are disjoint, because all components that are produced in the same iteration of the while loop are disjoint by construction of the algorithm and we can see that if A is a subset of B , then $L(A) < L(B)$, because A is produced by splitting B .

Moreover, we can see that each level one component has cost at least ϵ , because it is yet to be split by the algorithm. Thus for $i \geq 1$, each component has cost at least $(3/2)^{i-1}\epsilon$, as it has to be split $i-1$ times for some component to get to level 1, when it will have cost at least ϵ . And by Theorem 2.1.2, each time its cost will be $2/3$ of cost before splitting.

Now let's observe that because the cost of whole graph G is at most 1, then the total number of components at level i is at most $1/((3/2)^{i-1}\epsilon) = (2/3)^{i-1}/\epsilon$. In particular, the maximum level k of the component must satisfy

$$1 \leq (2/3)^{k-1}/\epsilon \leq (2/3)^{k-1}n$$

So we can derive from that $k \leq 1 + \log_{3/2} n$. We can then see that the height of the recursion tree of the algorithm is $O(\log n)$ and thus the algorithm will run in $O(n \log n)$ time.

Now we need to prove the bound on the size of C . Let $K_1, \dots, K_m$ be the components on some level $i \geq 1$ of sizes $n_1, \dots, n_m$ respectively. When splitting each of these components, we add at most $2\sqrt{2} \sum_{j=1}^m \sqrt{n_j}$ vertices to C by Theorem 2.1.2. We can recall now the bound m by the number of components at i -th level, namely: $m \leq (2/3)^{i-1}/\epsilon$. We trivially also have $\sum_{j=1}^m n_j \leq n$. To obtain the higher bound on $|C|$ we want to maximize the sum $\sum_{j=1}^m \sqrt{n_j}$ subject to constraint $\sum_{j=1}^m n_j \leq n$. For some fixed level m that sum is maximized by setting $n_j = n/m$. Now we can find a bound for the number of vertices added to C on each level:

$$2\sqrt{2} \sum_{j=1}^m \sqrt{n_j} \leq 2\sqrt{2} \sum_{j=1}^m \sqrt{n/m} = 2\sqrt{2} \sqrt{nm} \leq 2\sqrt{2} (2/3)^{(i-1)/2} \sqrt{n/\epsilon}$$

Now we can bound $|C|$:

$$|C| \leq \sqrt{n/\epsilon} \sum_{i=1}^{\infty} 2\sqrt{2} (2/3)^{(i-1)/2}$$

We know that the series converges to some constant as it is just a geometric series with $|q| < 1$, which means that $|C| = O(\sqrt{n/\epsilon})$ $\square$

By setting cost of each vertex in the graph to $1/n$ we obtain the following result:

Corollary 2.2.1.1. [7] *Let G be a planar graph with n vertices and let $0 < \epsilon < 1$. Then there exists some set C of $O(\sqrt{n/\epsilon})$ vertices such that removal of C from G leaves no connected component with more than ϵn vertices. Moreover, such a set can be found in $O(n \log n)$ time.*

Now we will introduce a class of problems described as **maximum induced subgraph problem** P on graph G with some graph property Q . The problem asks for a maximum set S of V that induces a subgraph with property Q [7]. Following just proved Lemma 2.2.1 we can deduce an algorithm (Algorithm 3, which we will call MISPP from now on). We can clearly observe that the algorithm halts and successfully solves the problem P .

Algorithm 3 MISPP algorithm

input: G : planar graph, $\epsilon \in (0, 1)$

output: S : subset of V that induces a subgraph with property Q

- 1: Apply Algorithm 2 to G to find separator C
 - 2: Find connected components of $G - C$ with maximum size ϵn
 - 3: **for all** connected components G_i of $G - C$ **do**
 - 4: Find a maximum set S_i inducing a subgraph with property Q by checking every subset of vertex set V for graph G_i
 - 5: **end for**
 - 6: Return $S \leftarrow \bigcup S_i$
-

Now we will introduce a lemma, that shows an upper bound on time complexity of Algorithm 3.

Lemma 2.2.2. [7] *The approximation MISPP Algorithm (Algorithm 3) has time complexity $O(\max\{n \log n, n2^{\epsilon n}\})$ and relative error of $O(1/\sqrt{\epsilon n})$ if the following conditions are satisfied:*

1. *The subgraph of G induced by S satisfies property Q .*
2. *The error $|S^*| - |S|$ is bounded by the number of vertices of C , that is, $|S^*| - |S| = O(|C|)$, where S^* is a maximum vertex set inducing a subgraph of G with Q .*
3. *$|S^*|$ is a positive fraction of n .*
4. *Property Q is recognizable in linear time (that is, one can determine in linear time whether a graph satisfies Q or not).*

Proof. **Time complexity**

Algorithm MISPP has two main steps that affect the time complexity. The first step is simply $O(n \log n)$ by Corollary 2.2.1.1. In step two, for each connected component $G_i = (V_i, E_i)$ we find S_i by checking every subset. Let

$n_i = |V_i|$, then we can find S_i in $O(n_i 2^{n_i})$ time (because from condition 4, we know that property Q is recognizable in linear time). So the total time is

$$O\left(\max\left\{\sum_{i=1}^n n_i 2^{n_i} : \sum_{i=1}^n n_i = n \wedge 0 \leq n_i \leq \epsilon n\right\}\right) = O\left(\frac{n}{\epsilon n} \epsilon n 2^{\epsilon n}\right) = O(n 2^{\epsilon n})$$

First equality stems from calculus and the fact that function $x2^x$ is strictly convex, so to maximize the value of the sum, we shall assign $n_i = \epsilon n$. We can see from both of those steps that the total time complexity has to be $O(\max\{n \log n, n 2^{\epsilon n}\})$.

Relative error

We shall define S^* as the maximum set of V that induces a subgraph satisfying property Q and S as the set produced by algorithm. We can see that condition 1 guarantees that we can compose a global solution S as a union of S_i , because each subgraph of G induced by S_i satisfies property Q . From condition 2 we shall see that $|S^*| - |S| = O(|C|) = O(\sqrt{n}/\epsilon)$. Combining that with condition 3 that guarantees us $|S^*| = \Omega(n)$ and by performing some simple operations on asymptotic functions we get that:

$$\frac{|S^*| - |S|}{|S^*|} \leq c_1 \sqrt{\frac{n}{\epsilon}} \cdot \frac{c_2}{n} = c_1 c_2 \sqrt{\frac{1}{\epsilon n}} = O(1/\sqrt{\epsilon n})$$

Which concludes the proof of Lemma 2.2.2. □

Chapter 3

Applications of planar separator theorem

3.1 Matching algorithm

In this section we will discuss an efficient algorithm for finding a maximum matching in a planar graph. The maximum matching problem asks for a maximum number of pairwise nonadjacent edges in a graph.

For bipartite graphs, one of the most known algorithms is the Hopcroft-Karp algorithm [16], which finds the maximum matching in $O((m+n)\sqrt{n})$ time by repeatedly finding a maximal set of vertex-disjoint shortest augmenting paths. For general graphs, the problem becomes more difficult due to presence of odd cycles. Edmonds introduced the blossom algorithm [13], which handles such cycles by contracting them during the search for augmenting paths and finds maximum matchings in $O(n^3)$. This approach was later improved by Micali and Vazirani, who developed an $O(m\sqrt{n})$ algorithm [21] for maximum matching in general graphs.

For planar graphs, we have already established that $m = O(n)$, so the best algorithms produce the maximum matching in $O(n^{3/2})$ time. A faster approach was proposed by Mucha and Sankowski who use an algebraic algorithm [22] based on Gaussian elimination that can compute the maximum matching in $O(n^{\omega/2})$ time, where ω is the exponent of matrix multiplication (as of 2026 the best known bound is $\omega < 2.371177$ [12]).

3.1.1 Algorithm

Although there exist exact algorithms to find a maximum matching for planar graphs in $O(n\sqrt{n})$ time, we can based on theorems recalled in Chapter

2 exhibit an approximation algorithm that runs in $O(n \log n)$ time and whose worst-case ratio is $1 - O(1/\sqrt{\log \log n})$.

One can find it interesting why we cannot use Algorithm MISPP directly and have worst-case ratio and running time guaranteed by Lemma 2.2.2. That is because the third property is not satisfied. The counter example is graph $K_{1,n-1}$ whose matching size is 1 regardless of the size of the graph, and that property demands that the size of the matching is a positive fraction of n .

It is also really important to see that even though MISPP algorithm was defined for problems with sets of vertices it easily extendable to sets of edges which we can employ in this section. However it is worth noting that the MISPP algorithm is a subroutine of Algorithm 4 ran on a reduced subgraph.

Theorem 3.1.1. *The approximation algorithm MATCHING for the maximum matching problem on planar graphs has running time $O(n \log n)$ and worst-case approximation ratio $1 - O(1/\sqrt{\log \log n})$.*

3.1.2 Correctness proof

We will start the proof to Theorem 3.1.1 by proving that it returns a matching with stated worst-case ratio. For that we will split the proof into two parts. First we will show that reductions performed by Algorithm 4 make the approximation error smaller than that of the graph before a reduction. And secondly, we will show that in base cases the algorithm returns the correct worst-case ratio. We will define the maximum matching of G as $M^*(G)$ and the matching produced by algorithm as $M(G)$. We will show that

$$\frac{|M(G)|}{|M^*(G)|} \geq 1 - O\left(\frac{1}{\sqrt{\log \log n}}\right)$$

Reductions

1. G has a vertex of degree 0. Let $G' = G - v$. Then we have that $M(G) = M(G')$, as there is no edge adjacent to v . For the same reason we have $M^*(G) = M^*(G')$, so the reduction does not affect the ratio.
2. G has a vertex of degree 1. Let u be the vertex adjacent to v in G and $G' = G - \{u, v\}$. If $M(G')$ is a matching produced by the algorithm then clearly $|M(G)| = |M(G')| + 1$ as we can add (u, v) to the matching. Also since $M^*(G)$ has exactly one edge adjacent to u

Algorithm 4 MATCHING procedure

input: G : planar graph, $M(G)$: matching

output: $M(G)$: matching

```
1:  $N \leftarrow |V(G)|$ 
2:  $v \leftarrow \min_{u \in V(G)} \deg(u)$ 
3: if  $N \leq \log \log n$  then
4:   Find exact matching  $M^*(G)$  using any reasonable algorithm
5:    $M(G) \leftarrow M^*(G)$ 
6: else if  $\deg(v) < 3$  then
7:   if  $\deg(v) = 0$  then
8:      $G' \leftarrow G - v$ 
9:     MATCHING( $G'$ ,  $M(G')$ )
10:     $M(G) \leftarrow M(G')$ 
11:   else if  $\deg(v) = 1$  then
12:      $u \leftarrow$  vertex adjacent to  $v$ 
13:      $G' \leftarrow G - \{u, v\}$ 
14:     MATCHING( $G'$ ,  $M(G')$ )
15:      $M(G) \leftarrow M(G') \cup \{(v, u)\}$ 
16:   else if  $\deg(v) = 2$  then
17:      $u, w \leftarrow$  neighbors of  $v$ 
18:      $G' \leftarrow$  graph made from  $G$  by identifying  $v, u$  and  $w$ 
19:     MATCHING( $G'$ ,  $M(G')$ )
20:     if  $M(G')$  does not have an edge adjacent to  $u$  in  $G$  then
21:        $M(G) \leftarrow M(G') \cup \{(v, u)\}$ 
22:     else
23:        $M(G) \leftarrow M(G') \cup \{(v, w)\}$ 
24:     end if
25:   end if
26: else
27:   Run Algorithm MISPP with  $\epsilon = \log \log n / N$  on  $G$ 
28:    $M(G) \leftarrow$  result of algorithm
29: end if
30: return  $M$ 
```

then by removing it we can reduce the matching by at most one, so we have $M^*(G) \leq M^*(G') + 1$. Now we can derive:

$$\frac{|M(G)|}{|M^*(G)|} \geq \frac{|M(G')| + 1}{|M^*(G')| + 1} \geq \frac{|M(G')|}{|M^*(G')|}$$

where the last equality stems from properties of rational numbers.

3. G has a vertex of degree 2. Let u, w be vertices adjacent to v and G' be a graph obtained from G by identifying those three vertices into single vertex v' . Let $M(G) = M(G') \cup \{(u, v)\}$ if no edge adjacent to u in G was selected to $M(G')$. Otherwise let $M(G) = M(G') \cup \{(w, v)\}$. Clearly we have $|M(G)| = |M(G')| + 1$. Now if the matching $M^*(G)$ contains either (v, u) or (v, w) , let's say (v, u) then by removing it by contraction to v' , $M^*(G')$ is a correct matching of size at most one less than $M^*(G)$. In the other case if both u and w were incident to edges in $M^*(G)$, then the matching $M^*(G')$ has to select one of them, so the property is conserved. In both cases we have $|M^*(G)| \leq |M^*(G')| + 1$. So again

$$\frac{|M(G)|}{|M^*(G)|} \geq \frac{|M(G')| + 1}{|M^*(G')| + 1} \geq \frac{|M(G')|}{|M^*(G')|}$$

We can see that all of the reductions create a graph if a worst-case ratio no worse than the original graph G .

Base cases

We have two base cases to discuss here:

1. If $N \leq \log \log n$, so we reduced our graph to a small fraction of the original one, we apply an exact matching algorithm, so $\frac{|M(G)|}{|M^*(G)|} = 1$ (if the denominator is 0, then numerator too, so by convention we set this ratio to 1)
2. If $N \geq \log \log n$ and $\forall_{v \in V(G)} \deg(v) \geq 3$ we apply MIS algorithm. To prove its correctness we have to check conditions from Lemma 2.2.2:
 - (a) Satisfied, as $M(G)$ is a matching so a subgraph induced by it satisfies the property that no two edges can have a common vertex
 - (b) Let $M^*(G)$ be the maximum matching. By removing vertices of a separator C at most $|C|$ edges from the matching have incident vertices in C . So in the induced graph $G - C$ the maximum

matching is of size at least $M^*(G) - |C|$. And this is also the size of $M(G)$ as we run the exact algorithm for matching here. So $|M(G)| \geq M^*(G) - |C|$, by rearranging we have $|M^*(G)| - |M(G)| \leq |C| = O(|C|)$, so the error is bound by size of C .

- (c) Here we will use the theorem from [23] which guarantees that if G is planar and has minimum degree of at least 3, then $|M^*(G)| \geq N/3$, so the optimal solution is a positive fraction of N .
- (d) We can easily check in linear time if the set found by the algorithm is the matching

We can see that all the conditions are satisfied, so by setting $\epsilon = \log \log n / N$ we have the desired worst-case error by Lemma 2.2.2.

3.1.3 Time complexity proof

At first we should discuss the time complexity of reductions. By keeping a graph as adjacency lists, so that each vertex not only has a previous reference and next reference but also the cross reference to other linked lists, we can perform a deletion in $O(\deg(v))$. And as each vertex is deleted only once, we can do that in amortized time $O(1)$. Another operation is identification of vertices which can be performed in total time $O(n \log n)$ [7]. To do that we keep an additional adjacency matrix and rewiring edges from the vertex of a smaller degree to one of higher. As each vertex is present in $O(\log n)$ such rewirings, we get the needed time complexity. To prove that, let's consider two vertices: v, u , such that $\deg(v) \geq \deg(u)$. It means that after rewiring edges from u to v , we have $\deg_1(v) \geq \deg_0(u)$, where subscript denotes the number of iteration of rewirings. So after k rewirings we have: $\deg_k(v) = 2^k \deg(u)$, and because $2^k \leq n$, then $k \leq \log_2(n)$. So each edge is rewired at most $O(\log n)$ times.

Now we can look at running time of base cases. If $N \leq \log \log n$ we use an exact algorithm to find the matching. If its time complexity is at most $O(n2^N)$, then it is enough for our desired time complexity. And if $N \geq \log \log n$, then from Lemma 2.2.2 we have guaranteed

$$O(\max\{N \log N, N2^{\epsilon N}\}) = O(N \log n) = O(n \log n)$$

3.2 Vertex cover algorithm

Another problem we will look at is Vertex Cover Problem which is NP-complete even for planar graphs [8, 14]. In [7] we can find an algorithm

very similar to the one for matching problem discussed in a previous section. However for Vertex Cover Problem another interesting approach may be found in [2] which will be shown in this section. At first we shall define what a vertex cover is.

Definition 3.1 (vertex cover, [9]). *Let $G = (V, E)$ be a graph. We call $VC \subseteq V$ a cover of E (or vertex cover of G) if every edge of G is incident with a vertex in VC . This problem is $\mathbb{NP}$ -complete.*

Vertex Cover Problem asks for a minimum vertex cover in G . Another related problem is the Independent Set Problem that asks for a maximum independent set.

Definition 3.2 (independent set, [9]). *Let $G = (V, E)$ be a graph. We call $I \subseteq V$ an independent set if no two of its elements are adjacent. This problem is $\mathbb{NP}$ -complete.*

It is clear that those problems are complementary, so solving Independent Set Problem gives a solution to Vertex Cover Problem as $VC \cup I = V$.

3.2.1 Algorithm

In [20] authors give an approximation algorithm (which is essentially MISAP algorithm discussed in Chapter 2) to find a maximum independent set in $O(n \log n)$ time with relative error $O(1/\sqrt{\log \log n})$, which means that for some constant c :

$$\frac{|I^*| - |I|}{|I^*|} \leq \frac{c}{\sqrt{\log \log n}}$$

But we know that $|VC^*| + |I^*| = n$, so after some simple arithmetics:

$$\frac{|VC| - |VC^*|}{|VC^*|} \leq \frac{c(n/|VC^*| - 1)}{\sqrt{\log \log n}}$$

This means that in order to use approximation scheme from [20] for independent set to solve VC Problem we will have to show that $n/|VC^*|$ converges to some $\epsilon_0 > 1$.

To do that we will have to pre-process graph G and apply approximation scheme afterwards. At start we shall start with a few definitions introduced by authors of [2].

Definition 3.3 (rosary, [2]). *Let $G = (V, E)$ be a simple planar graph. A **rosary** in G is a cycle of even length $2l$, denoted by $v_1 - v_2 - \dots - v_{2l} - v_1$, such that the degree of every even-indexed vertex is exactly two for all $i = 1, 2, \dots, l$.*

Definition 3.4 (bead, [2]). Let $G = (V, E)$ be a simple planar graph. A vertex $v \in V$ of degree two is called a **bead** if it plays the role of an even-indexed vertex v_{2i} in some rosary in G .

Definition 3.5 (proper graph, [2]). Let $G = (V, E)$ be a simple planar graph. We will call it **proper** if the degree of all vertices is at least 2 and the set Y_B of all its beads satisfies $|Y_B| < 1/6|V|$.

The authors prove a very useful theorem regarding proper graphs.

Theorem 3.2.1. [2] Let $G = (V, E)$ be a simple planar graph with all vertices having degree at least 2. Then we have:

$$|VC^*| > \frac{1}{5}(|V| - |Y_B|).$$

. If G is proper then by Theorem 3.2.1 we have $|V|/|VC^*| < 6$. And that is exactly what we need to apply the approximation scheme. So the preparatory algorithm from [2] has to reduce G to a proper graph. The general idea of that algorithm is to at first get rid of all vertices with degree less than 2 and later reduce the size of set of beads. We shall see exact steps in listing for Algorithm 5.

Algorithm 5 Preparatory algorithm

```
1: procedure PREP( $G$ )
2:    $VC \leftarrow \emptyset$ 
3:    $U \leftarrow \emptyset$ 
4:   while true do
5:     while exists  $v \in V(G[V \setminus U])$  with  $\deg(v) < 2$  do
6:       if exists  $v \in V \setminus U$  with  $\deg_{G[V \setminus U]}(v) = 0$  then
7:         Add  $v$  to  $U$ 
8:       end if
9:       if exists  $v \in V \setminus U$  with  $\deg_{G[V \setminus U]}(v) = 1$  then
10:        Let  $(v, u) \in E(G[V \setminus U])$  be the edge of  $v$ 
11:        Add  $v, u$  to  $U$  and  $u$  to  $VC$ 
12:      end if
13:    end while
14:    while there are simple isolated circuits in  $G[V \setminus U]$  do
15:      Let  $v_1, \dots, v_k, v_1$  be a simple isolated circuit in  $G[V \setminus U]$ 
16:      for  $i = 1, \dots, k$  do
17:        if  $i$  is odd then
18:          add  $v_i$  to  $VC$ 
19:        end if
20:      end for
21:      add  $v_i$  to  $U$ 
22:    end while
23:     $B(X, Y, E') \leftarrow \text{BIPARTITE}(G[V \setminus U])$ 
24:    if  $|Y| \geq \frac{1}{6}|V \setminus U|$  then
25:       $VC \leftarrow VC \cup X$ 
26:       $U \leftarrow U \cup X \cup Y$ 
27:    end if
28:    if  $|Y| < \frac{1}{6}|V \setminus U|$  or  $V \setminus U = \emptyset$  then
29:      break
30:    end if
31:  end while
32: end procedure
```

3.2.2 Correctness of the algorithm

We shall at first prove that Algorithm 5 outputs a minimum vertex cover for $G[U]$ and an induced subgraph $G[V \setminus U]$ which shall be a proper graph. The algorithm has three phases that can be extracted: deleting small degree vertices, deleting isolated cycles, construction of an auxiliary bipartite graph.

The first phase is deleting all the vertices with vertex degree less than 2. We can see that the that the covering of deleted induced subgraphs is

optimal.

- If $\deg(v) = 0$, then it does not have an incident edge to it, so the algorithm does not take it into the vertex cover which is optimal for the minimum vertex cover.
- If $\deg(v) = 1$, then it has to be picked to a minimum vertex cover or its adjacent vertex u to cover the incident edge e . The algorithm here makes an optimal choice one again of picking u to the vertex cover, as it may have more incidents than only e as opposed to v .

The second phase is deleting all of the isolated cycles. It is easy to see that here the algorithm makes an optimal decision, as it adds to the vertex cover all the vertices with odd indices. It is the optimal decision, as the cycle has exactly k edges to cover, so the minimum number of vertices to pick to cover the cycle $\lceil k/2 \rceil$, which is achieved by picking all the odd indices.

Now we shall focus on the third part of the algorithm, which is the construction of a bipartite graph from the residual graph $G[V \setminus U]$. The algorithm picks the whole vertex set X to VC . To prove that it is optimal we have to show:

Lemma 3.2.2. *Let $B(X, Y, E)$ be a bipartite graph in which for every $x \in X$, $\deg(x) \geq 2$ and for every $y \in Y$, $\deg(y) \leq 2$.*

Proof. The proof follows by induction on $|Y|$. For the base case $|Y| = 0$, we have that $|X| = 0$ (because the degree for each vertex of x has to be at least 2) and $E = \emptyset$ which trivially holds.

Now for induction step suppose that the theorem holds for all $|Y| < m$, and let $B(X, Y, E)$ be such a bipartite graph that $|Y| = m$. Let y the vertex in Y with minimum degree.

If $\deg(y) = 0$ then delete y from B and the lemma is true by induction on the reduced graph, so it is true for the original graph as well.

If $\deg(y) = 1$, let x be the adjacent vertex from X . Remove x , y and all their incident edges. The set $X \setminus \{x\}$ is therefore the minimum vertex cover of the bipartite graph $B(\{X \setminus x\}, \{Y \setminus y\}, E')$ by induction. So for the original graph add x to the vertex cover, so that it covers all the removed incident edges. By doing that we have that X is the minimum vertex cover of the original graph.

If $\deg(y) = 2$, then all vertices in $B(X, Y, E)$ have degrees at least 2, so there are cycles. Pick any simple cycle C . All the vertices from C that are in X form the minimum vertex cover for that cycle. Remove C and all the incident edges. The minimum vertex cover for $B(X', Y', E')$ is X' by induction, so with addition of the deleted vertices from C that formed the

minimum vertex cover for C we have that X is the minimum vertex cover for the original graph $B(X, Y, E)$. $\square$

Algorithm 6 BIPARTITE procedure

input: H : planar graph
output: B : bipartite graph

- 1: Label all $v : \deg(v) > 2$ with "+"
- 2: Add those vertices to queue Q
- 3: **while** Q not empty **do**
- 4: $v \leftarrow Q.popFront()$
- 5: Let L be all unlabelled vertices adjacent to v in H
- 6: **for all** $v_L \in L$ **do**
- 7: label opposite to v (using "+" or "-")
- 8: $Q.add(v_L)$
- 9: **end for**
- 10: **end while**
- 11: $X \leftarrow \{v : v \in V(H) \text{ and labelled "+"}\}$
- 12: $Y \leftarrow \{v : v \in V(H) \text{ and labelled "-"}\}$
- 13: $E' \leftarrow \{(u, v) : u \in X, v \in Y, (u, v) \in E(H)\}$
- 14: Let $B = (X, Y, E')$ be a bipartite graph
- 15: **while exists** $v \in B : \deg_B(v) < 2$ **do**
- 16: Remove v and incident edges
- 17: **end while**
- 18: **return** B

Now we can see that by construction from the Algorithm 6, all of the vertices with degree greater than 2 land in set X of the bipartite graph, so by Lemma 3.2.2 X is the minimum vertex cover of $B(X, Y, E)$. That is because each vertex with degree greater than 2 is labeled as +. This opposite labelling construction serves as a natural separation of vertices into two sets, where after carefully selecting appropriate edges, we naturally get sets of independent vertices to construct a bipartite graph.

Now we can observe that if Algorithm 5 halts with $VU = \emptyset$, then by the arguments that were just established, VC is the minimum vertex cover of the whole graph, and the residual graph is empty. On the other hand if the algorithm halts with the condition $|Y| < 1/6|V \setminus U|$, then set VC is the minimum vertex cover of $G[U]$. Moreover the degrees of all of the vertices is at least 2. The only thing that is left to prove is that the set of beads Y_B satisfies $|Y_B| < 1/6|V \setminus U|$.

Lemma 3.2.3. *If y is a bead, then $y \in Y$.*

Proof. Let $(y_1, x_1, \dots, y_l, x_l)$ be a rosary in H (input graph to Algorithm 6). Since H contains no isolated circuits as a result of earlier steps for Algorithm 5, some of the x vertices are of degree greater than 2 and are labelled $+$ in Algorithm 6. All the other vertices in a rosary are of degree 2 (so their neighbors are only other vertices from the rosary) and therefore their labelling cannot be affected by a vertex not from the rosary. It follows that every bead will finally get labelled $-$, so all of the beads will land in Y . $\square$

By this lemma we have that $Y_B \subseteq Y$, so $G[V \setminus U]$ is a proper graph.

3.2.3 Time complexity

We can prove the following lemma.

Lemma 3.2.4. *Time complexity of the preparatory Algorithm 5 is $O(n)$.*

Proof. Construction of the bipartite graph, so the running time of Algorithm 6 is $O(n)$ as this is just a BFS. Moreover time spent on all of the transformations like deletion of vertices and isolated circuits is also $O(n)$. Note that with every step of the outer while loop of Algorithm 5, the number of vertices in $V \setminus U$ is reduced by a factor of $5/6$. Let $n_i = |V \setminus U|$ at i -th step of the outer while loop. Let cn_i be the bound of the complexity of the i -th step. Since $n_i \leq (5/6)^i n$ then the total running time of Algorithm 5 is

$$\sum_{i=0}^{\infty} cn_i \leq \sum_{i=0}^{\infty} (5/6)^i n = 6 \cdot n \cdot c = O(n)$$

$\square$

Chapter 4

Benchmarks and implementation details

4.1 Implementation details

4.1.1 KOALA NetworKit

All of the algorithms described in this paper were implemented using C++20 for an open-source KOALA NetworKit [25] library.

KOALA NetworKit joins two open-source libraries: KOALA and NetworKit library. At first we can get familiar with them by following README file from KOALA NetworKit repository [25].

- NetworKit [24] is an open-source general purpose networking analysis and graph mining library written in C++. Graphs are represented in a compact form, while preserving the efficiency of changes to their structure such as node and edge additions and deletions. The library contains algorithms mostly for the structural analysis of massive complex real-world networks, but lacks some of the most important general graph algorithms like graph coloring.
- In places where NetworKit library is lacking, KOALA [11] library excels. It is an open-source library of C++ templates, developed at Gdansk University of Technology Department of Algorithms and System Modeling. Its main part consists of an implementation of a broad set of procedures in algorithmic graph theory and network problems in discrete optimization. The library contains algorithms for problems like: graph coloring, graph search, flow problems, graph recognition for graph families and many more.

Because KOALA NetworKit is a fork of KOALA library built on structures provided by NetworKit, it combines the best of two worlds by providing a simple, fast and powerful unified interface for a broad class of algorithms.

The contribution of this work to the KOALA NetworKit library are mainly in the following files:

- `cpp/techniques/separator/LiptonTarjanPlanarSeparator.cpp` and `cpp/techniques/separator/LiptonTarjanFundamentalCycle.cpp` provide an efficient $O(n)$ implementation of Lipton-Tarjan planar separator algorithm discussed in Section 2.1
- `cpp/techniques/separator/MISP.cpp` provides the implementation of Algorithm 2 and Algorithm 3 for further use in other algorithms
- `cpp/vertex_cover/PlanarVertexCover.cpp` provides the implementation of approximation algorithm for Vertex Cover Problem (Algorithm 5 and Algorithm 6)
- `cpp/matching/PlanarMaximumMatching.cpp` provides the implementation of approximation algorithm for Maximum Matching Problem (Algorithm 4)

4.1.2 Lipton-Tarjan planar separator algorithm

Implementation of this algorithm uses an embedding algorithm provided by KOALA NetworKit library that is based on Boyer-Myrvold $O(n)$ embedding algorithm from Boost [3] library. The critical section of this algorithm which is Algorithm 1 is based on two `EdgeScanner` structures that allows for scanning edges in two directions of the cycle one at a time, which is the cornerstone of $O(n)$ implementation. Moreover, in order to update the cycle structure in $O(1)$ time at the of each iteration, it is implemented as a double linked list to ensure quick splicing and merging.

4.1.3 MISP algorithm

This algorithm is implemented as a generic class `MISP<T>`, so that it supports both vertex and edge sets for Vertex Cover and Maximum Matching Problems respectfully. In order to achieve the theoretical $O(n \log n)$ time complexity, the graphs are compacted and remapped during each iteration of the algorithm using methods from KOALA NetworKit library.

4.1.4 Vertex Cover algorithm

This algorithm faithfully follows the pseudocode provided in listings for Algorithm 5 and Algorithm 6. It is worth noting, that to minimize the effect of graph compacting and mapping, it has been decided, to abandon it for this algorithm, which changes the time complexity of Algorithm 5 to $O(n \log n)$ instead of $O(n)$ guaranteed by theoretical analysis. However, it is not a major issue, as MISP already has $O(n \log n)$ time complexity which is a bottleneck for the algorithm.

4.1.5 Maximum Matching algorithm

Implementation of this algorithm follows the pseudocode provided in listing for Algorithm 4. However, to avoid problems with stack memory during recursive calls of the algorithm, it has been decided to abandon its recursive structure in favor of splitting the algorithm into two parts. The first one is reduction phase, where we keep track of vertices that take part in each reduction onto a separate stack vector. After reduction phase is finished, the exact solution is found using MISP algorithm. The final phase of the algorithm is backtracking and retrieving the matching using the stack.

4.2 Benchmark structure and results

The benchmarks have been run for two approximation algorithms: Vertex Cover algorithm and Maximum Matching algorithm, as well as a separate benchmark for Lipton-Tarjan planar separator algorithm itself. The benchmarks consisted of six planar graph families:

- cycle graphs - they represent cycles of length n
- grid graphs - square grids, with side lengths $\lfloor \sqrt{n} \rfloor$
- trigrid graphs - square grids, with side lengths $\lfloor \sqrt{n} \rfloor$ such that every internal face is a triangle
- hexagonal grid graphs - grids, with side lengths $\lfloor \sqrt{n} \rfloor$ such that every face is a hexagon (a cycle of length 6)
- tree graphs - random trees with n vertices, where each vertex is randomly (with uniform distribution) attached to some other vertex already present in a graph

- apollonian graphs - a maximal planar graph of n vertices that is created by following a simple algorithm

Algorithm 7 APOLLONIAN procedure

input: n : number of vertices

output: G : graph

- 1: $G \leftarrow$ triangle
 - 2: **while** $|V(G)| < n$ **do**
 - 3: Select a face $F = (u, v, w)$ uniformly at random
 - 4: Add a new vertex x to G
 - 5: Add edges (x, u) , (x, v) , and (x, w) to G
 - 6: **end while**
 - 7: **return** G
-

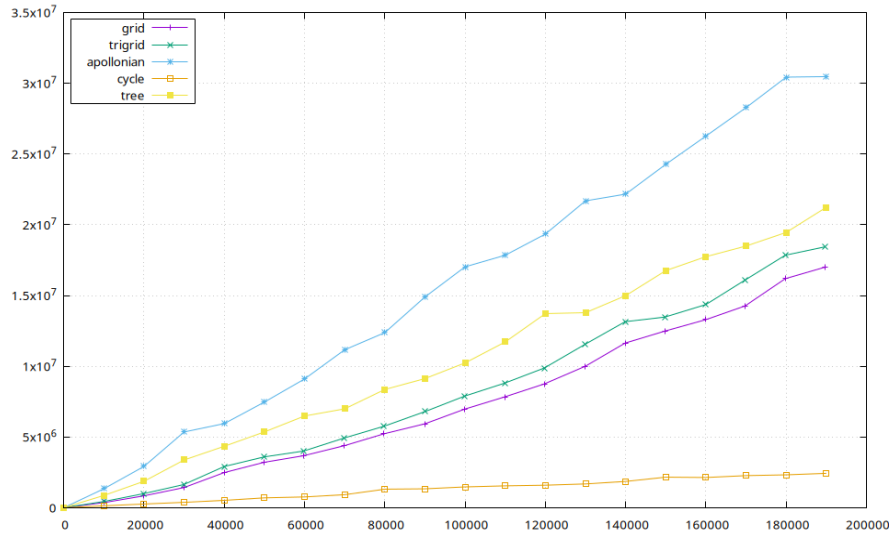


Figure 4.1: Lipton-Tarjan separator algorithm running time comparison

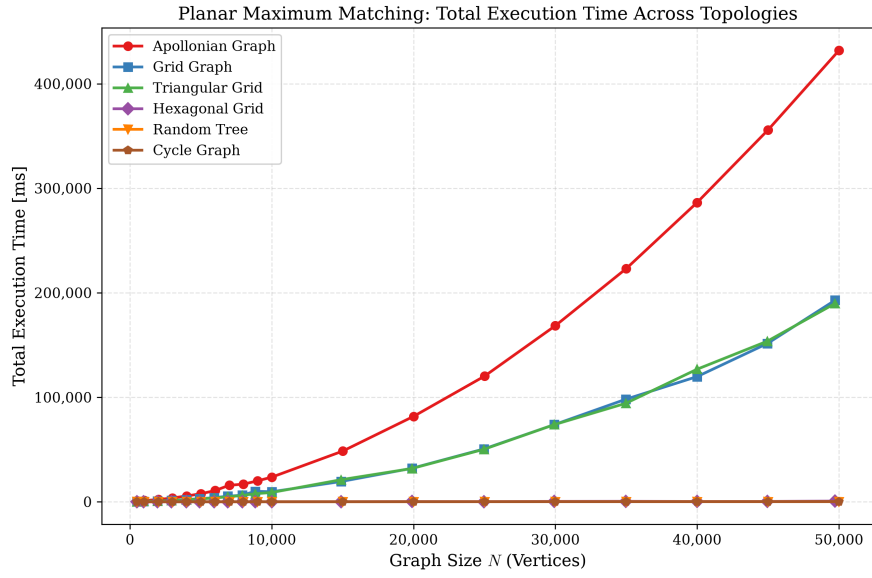


Figure 4.2: Matching algorithm total running time comparison

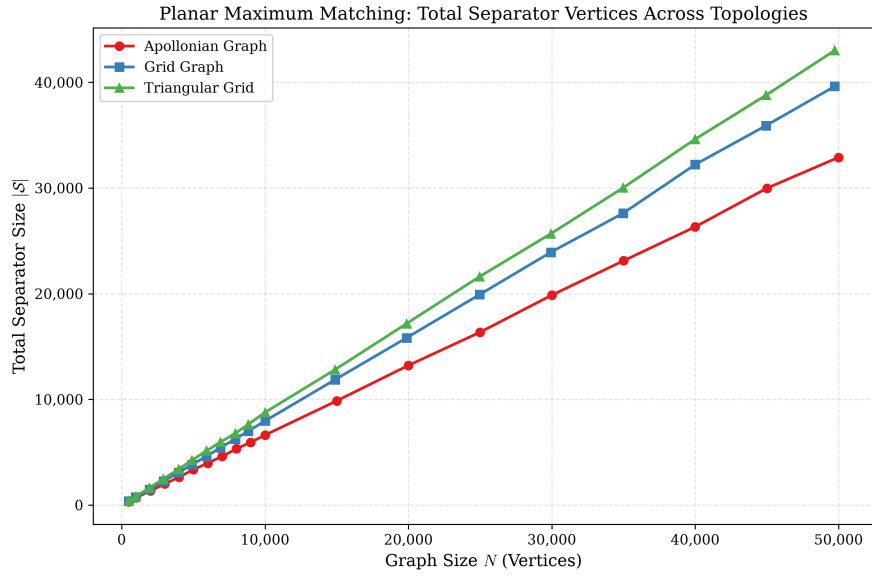


Figure 4.3: Matching algorithm separator size comparison

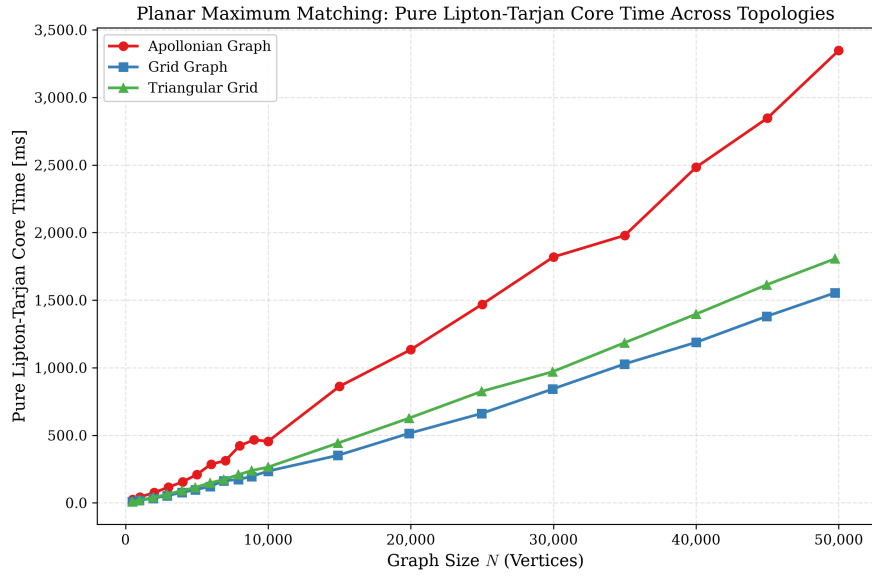


Figure 4.4: Matching algorithm, separator algorithm running time comparison

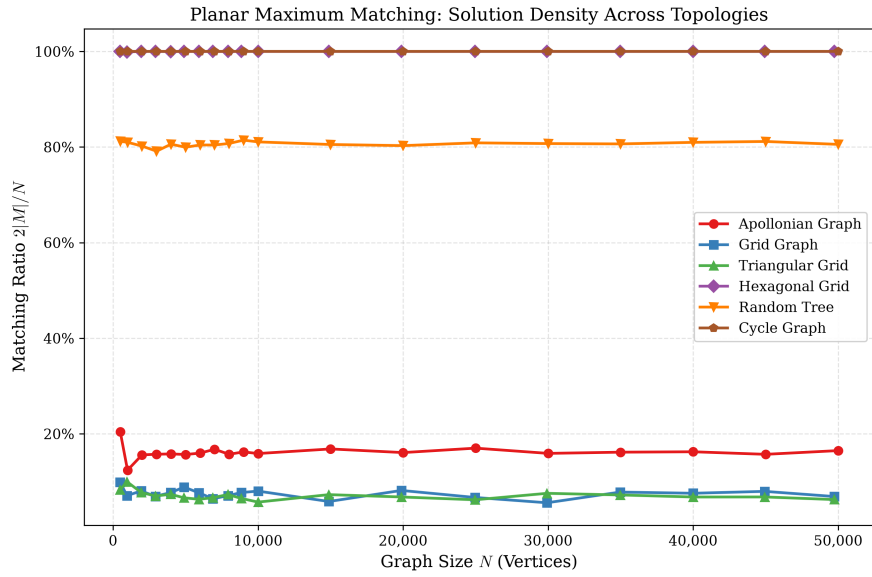


Figure 4.5: Matching algorithm percentage of node covered

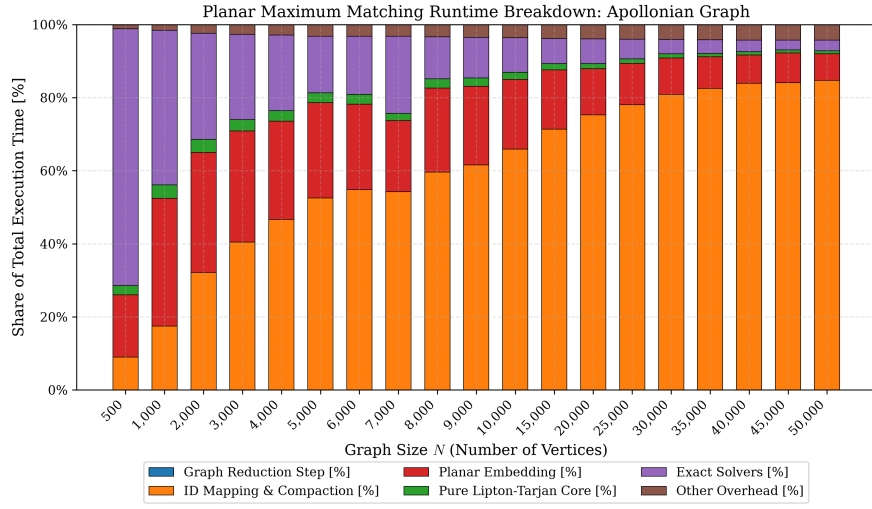


Figure 4.6: Matching algorithm percentage breakdown of different phases for apollonian graphs

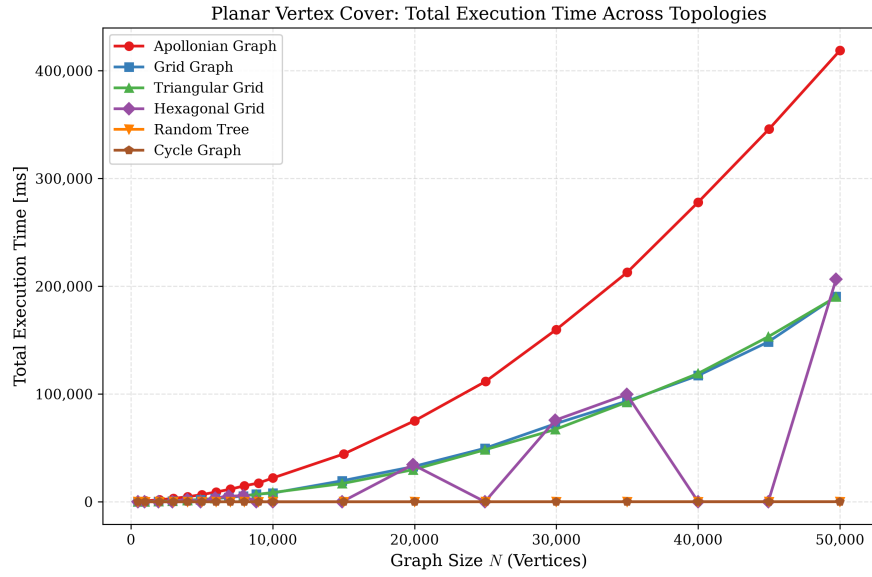


Figure 4.7: Vertex Cover algorithm total running time comparison

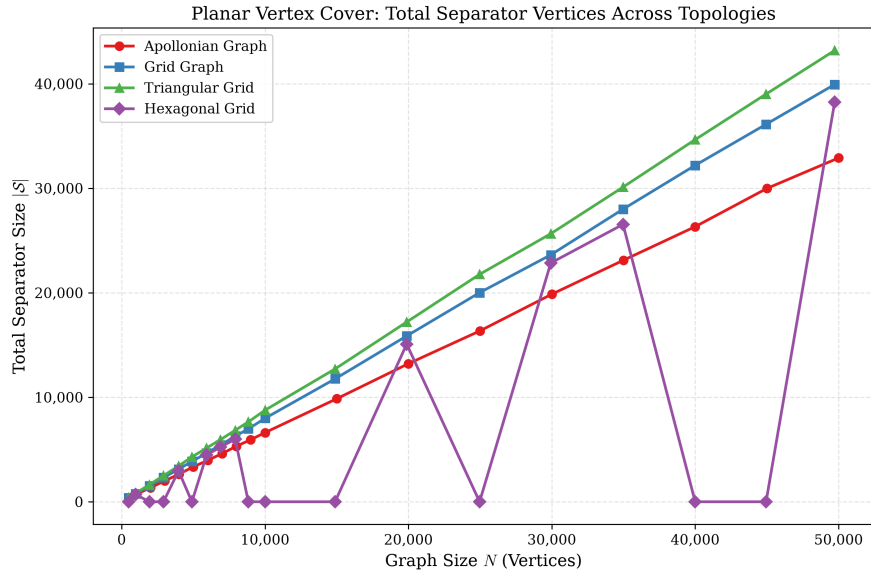


Figure 4.8: Vertex Cover algorithm separator size comparison

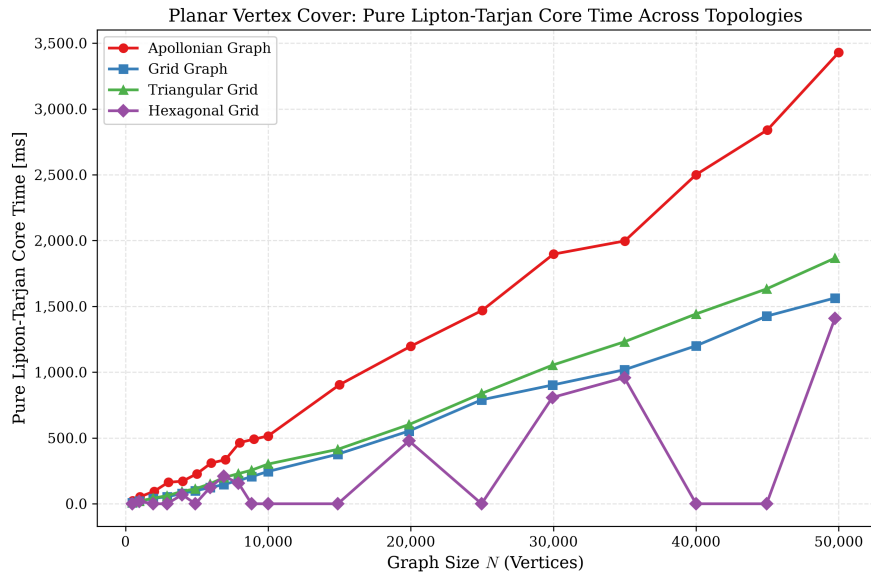


Figure 4.9: Vertex Cover algorithm, separator algorithm running time comparison

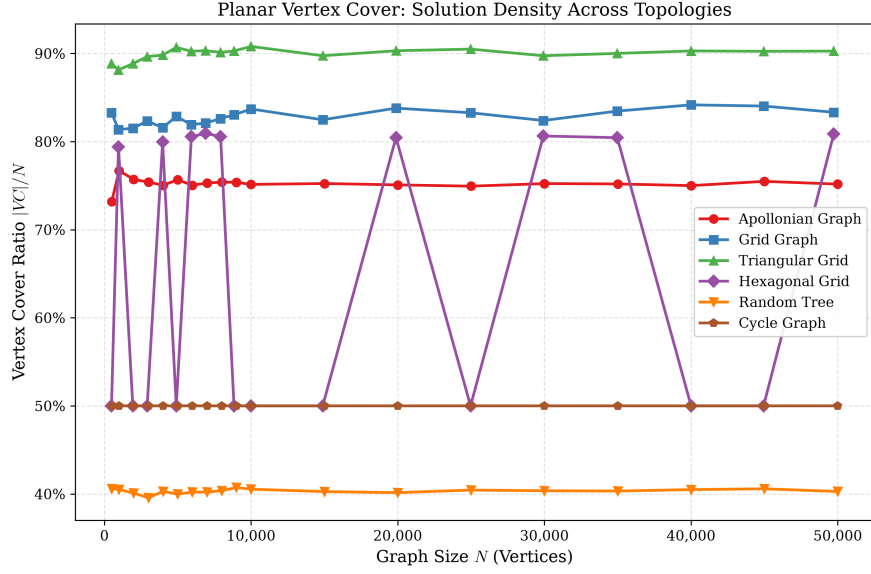


Figure 4.10: Vertex Cover algorithm percentage of node covered

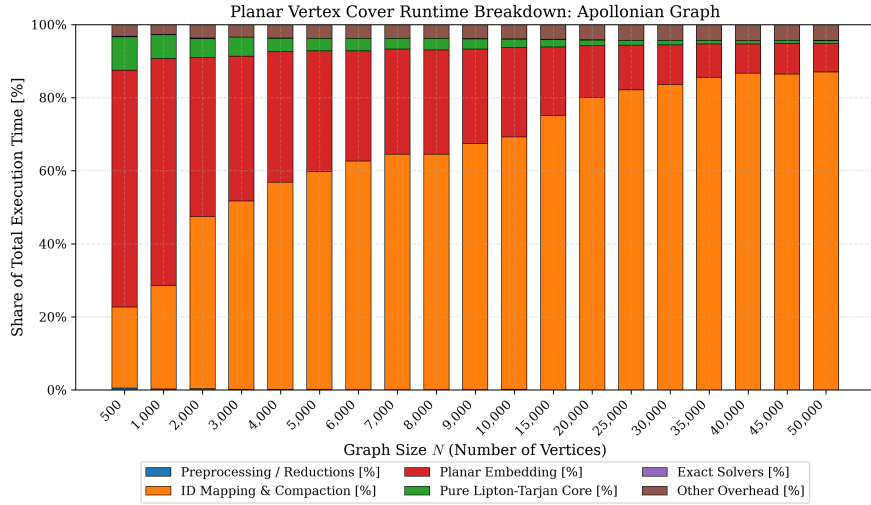


Figure 4.11: Vertex Cover algorithm percentage breakdown of different phases for apollonian graphs

From Figure 4.1 we can see that the total running time of isolated Lipton Tarjan planar separator algorithm is linear, that aligns perfectly with what is expected from theoretical perspective. Moreover, we can see that the size of a separator rises in linear manner (see Figure 4.3 and Figure 4.8). If we recall Algorithm 2 it guarantees a separator of size $O(n/\epsilon)$, and in the case of

our algorithms $\epsilon = \frac{\log \log n}{n}$ and because function $f(n) = \log \log n$ grows very slowly, for our small input we have linear progression. The next interesting thing is how much time mapping and planar embedding takes when the size of input grows. We can see exactly that in Figure 4.6 and Figure 4.11. This brings a huge overhead on the total running time of the algorithm which can be seen in Figure 4.7 and Figure 4.2. And for that the total running time of Lipton-Tarjan separator algorithms across all recursive calls has been measured, providing us with the real running time of the algorithm, which can be seen in Figure 4.4 and Figure 4.9.

At last, we can look at Figure 4.5 and Figure 4.10 which highlights how well does the approximation algorithm cover vertices and edges in respectful algorithms. Those plots highlight how difficult it is to measure theoretical approximation algorithms. As the approximation error of both algorithms is $O(\frac{1}{\sqrt{\log \log n}})$, for small sizes, that function is almost constant. Moreover as it had been mentioned in Section 3.1, by construction of MISP algorithm we lose all of the edges incident to vertices of a separator. And because the size of a separator increases linearly, for small graphs many potential matching edges are lost. A better approach for an approximation matching algorithm based on a planar separator can be found in [20].

Bibliography

- [1] Kenneth Appel and Wolfgang Haken. “The Solution of the Four-Color-Map Problem”. In: *Scientific American* 237.4 (1977), pp. 108–121. DOI: 10.1038/scientificamerican1077-108.
- [2] Reuven Bar-Yehuda and Shimon Even. “On Approximating a Vertex Cover for Planar Graphs”. In: *Proceedings of the 14th Annual ACM Symposium on Theory of Computing (STOC 1982)*. Ed. by Harry Lewis, Barbara Simons, Walter Burkhard, and Lawrence Landweber. Association for Computing Machinery, 1982, pp. 303–309. ISBN: 978-0-89791-070-5. DOI: 10.1145/800070.802205.
- [3] Boost Developers. *Boost C++ Libraries*. Version 1.92.0. Aug. 2026. URL: <https://www.boost.org/releases/1.92.0/> (visited on 09/14/2026).
- [4] Kellogg Booth and George Lueker. “Testing for the Consecutive Ones Property, Interval Graphs, and Graph Planarity Using PQ-Tree Algorithms”. In: *Journal of Computer and System Sciences* 13.3 (1976), pp. 335–379. DOI: 10.1016/S0022-0000(76)80045-1.
- [5] John Boyer, Pier Francesco Cortese, Maurizio Patrignani, and Giuseppe Di Battista. “Stop Minding Your P’s and Q’s: Implementing a Fast and Simple DFS-Based Planarity Testing and Embedding Algorithm”. In: *Proceedings of the 11th International Symposium on Graph Drawing (GD 2003)*. Ed. by Giuseppe Liotta. Vol. 2912. Lecture Notes in Computer Science. Springer, 2004, pp. 25–36. ISBN: 978-3-540-20831-0. DOI: 10.1007/978-3-540-24595-7_3.
- [6] Norishige Chiba, Takao Nishizeki, Shigenobu Abe, and Takao Ozawa. “A Linear Algorithm for Embedding Planar Graphs Using PQ-Trees”. In: *Journal of Computer and System Sciences* 30.1 (1985), pp. 54–76. DOI: 10.1016/0022-0000(85)90004-2.

- [7] Norishige Chiba, Takao Nishizeki, and Nobuji Saito. “Applications of the Lipton and Tarjan’s Planar Separator Theorem (Applied Combinatorial Theory and Algorithms)”. In: *RIMS Kôkyûroku* 427 (1981), pp. 148–164.
- [8] Thomas Cormen, Charles Leiserson, Ronald Rivest, and Clifford Stein. *Introduction to Algorithms*. 4th ed. The MIT Press, 2022. ISBN: 978-0-262-04630-5.
- [9] Reinhard Diestel. *Graph Theory*. 5th ed. Vol. 173. Graduate Texts in Mathematics. Springer, 2017. ISBN: 978-3-662-53621-6. DOI: 10.1007/978-3-662-53622-3.
- [10] Hristo Djidjev. “On the Problem of Partitioning Planar Graphs”. In: *SIAM Journal on Algebraic Discrete Methods* 3.2 (1982), pp. 229–240. DOI: 10.1137/0603022.
- [11] Tomasz Dobrowolski, Dariusz Dereniowski, and Łukasz Kuszner. “KOALA Graph Coloring Library: An Open Graph Coloring Library for Real-World Applications”. In: *Proceedings of the 1st International Conference on Information Technology (IT 2008)*. Ed. by Andrzej Stepnowski, Marek Moszyński, Thaddeus Kochanski, and Jacek Dabrowski. IEEE, 2008, pp. 1–4. ISBN: 978-1-4244-2244-9. DOI: 10.1109/INFTECH.2008.4621641.
- [12] Emilien Dupont et al. *Improving the Matrix Multiplication Exponent with Modern Optimization and AlphaEvolve*. 2026. arXiv: 2608.16884 [cs.DS]. URL: <https://arxiv.org/abs/2608.16884>.
- [13] Jack Edmonds. “Paths, Trees, and Flowers”. In: *Canadian Journal of Mathematics* 17 (1965), pp. 449–467. DOI: 10.4153/CJM-1965-045-4.
- [14] Michael Garey, David Johnson, and Larry Stockmeyer. “Some Simplified NP-Complete Graph Problems”. In: *Theoretical Computer Science* 1.3 (1976), pp. 237–267. DOI: 10.1016/0304-3975(76)90059-1.
- [15] Alan George. “Nested Dissection of a Regular Finite Element Mesh”. In: *SIAM Journal on Numerical Analysis* 10.2 (1973), pp. 345–363. DOI: 10.1137/0710032.
- [16] John Hopcroft and Richard Karp. “An $n^{5/2}$ Algorithm for Maximum Matchings in Bipartite Graphs”. In: *SIAM Journal on Computing* 2.4 (1973), pp. 225–231. DOI: 10.1137/0202019.
- [17] John Hopcroft and Robert Tarjan. “Efficient Planarity Testing”. In: *Journal of the ACM* 21.4 (1974), pp. 549–568. DOI: 10.1145/321850.321852.

- [18] Camille Jordan. “Sur les assemblages de lignes”. In: *Journal für die reine und angewandte Mathematik* 70 (1869), pp. 185–190. DOI: 10.1515/CRL.1869.70.185.
- [19] Richard Lipton and Robert Tarjan. “A Separator Theorem for Planar Graphs”. In: *SIAM Journal on Applied Mathematics* 36.2 (1979), pp. 177–189. DOI: 10.1137/0136016.
- [20] Richard Lipton and Robert Tarjan. “Applications of a Planar Separator Theorem”. In: *Proceedings of the 18th Annual Symposium on Foundations of Computer Science (FOCS 1977)*. IEEE Computer Society, 1977, pp. 162–170. DOI: 10.1109/SFCS.1977.6.
- [21] Silvio Micali and Vijay Vazirani. “An $O(\sqrt{|V|}|E|)$ Algorithm for Finding Maximum Matching in General Graphs”. In: *Proceedings of the 21st Annual Symposium on Foundations of Computer Science (FOCS 1980)*. IEEE Computer Society, 1980, pp. 17–27. DOI: 10.1109/SFCS.1980.12.
- [22] Marcin Mucha and Piotr Sankowski. “Maximum Matchings in Planar Graphs via Gaussian Elimination”. In: *Algorithmica* 45.1 (2006), pp. 3–20. DOI: 10.1007/s00453-005-1187-5.
- [23] Takao Nishizeki and Ilker Baybars. “Lower Bounds on the Cardinality of the Maximum Matchings of Planar Graphs”. In: *Discrete Mathematics* 28.3 (1979), pp. 255–267. DOI: 10.1016/0012-365X(79)90133-X.
- [24] Christian Staudt, Aleksejs Sazonovs, and Henning Meyerhenke. “NetworKit: A Tool Suite for Large-Scale Complex Network Analysis”. In: *Network Science* 4.4 (2016), pp. 508–530. DOI: 10.1017/nws.2016.20.
- [25] Krzysztof Turowski. *koala-networkkit*. KOALA library fork built on top of NetworKit. 2024. URL: <https://github.com/krzysztof-turowski/koala-networkkit> (visited on 09/14/2026).
- [26] Peter Ungar. “A Theorem on Planar Graphs”. In: *Journal of the London Mathematical Society* 26.4 (1951), pp. 256–262. DOI: 10.1112/jlms/s1-26.4.256.
- [27] Klaus Wagner. “Über eine Eigenschaft der ebenen Komplexe”. In: *Mathematische Annalen* 114.1 (1937), pp. 570–590. DOI: 10.1007/BF01594196.
- [28] David Williamson and David Shmoys. *The Design of Approximation Algorithms*. Cambridge University Press, 2011. ISBN: 978-0-521-19527-0. DOI: 10.1017/CB09780511921735.